Logistiklabb 2026

Rikard Ehrenbåge, David Andersson

November 10, 2026

Logistiklabb

Grupp: Rikard & David

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import linprog
pd.set_option("display.precision", 2)

from IPython.display import Markdown, display

def visa_tabell(df, format={}):
    # visar en tabell som markdown, så att den blir snygg både i notebooken och i PDF:en
    kolumner = list(df.columns)
    rader = ["| " + " | ".join(str(k) for k in kolumner) + " |", "|" + "---|" * len(kolumner)]
    for _, rad in df.iterrows():
        celler = []
        for k in kolumner:
            v = rad[k]
            if k in format:
                celler.append(format[k].format(v))
            elif isinstance(v, float):
                celler.append(f"{v:.2f}")
            else:
                celler.append(str(v))
        rader.append("| " + " | ".join(celler) + " |")
    display(Markdown("\n".join(rader)))
```

---

1. Lagerstyrning och produktion

1a) Exponentiell utjämning

Prognosen uppdateras med

$$u(t) = a\,d(t) + (1-a)\,u(t-1), \qquad 0 \le a \le 1,$$

där $u(t-1)$ är prognosen för månad t (gjord månaden innan) och $u(0) = 50$ är startskattningen (prognos för januari). Prognosen för december är alltså $u(\text{nov})$.

Därefter beräknades APE (Absolute Percentage Error) för varje månad, som visar hur stort felet var mellan det verkliga och prognosiserade värdet.

$$\text{APE}(t) = \frac{|d(t) - u(t-1)|}{d(t)} \cdot 100\,\%$$

När felet hade beräknats för varje måndad kunde ett medelvärde (MAPE) bestämmas.

Prognosfelet mäts med MAPE över de 11 månader där vi har utfall:

$$\text{MAPE} = \frac{100}{n} \sum_{t=1}^{n} \frac{|d(t) - u(t-1)|}{d(t)}\ \%.$$

Vikten a bestämmer hur stor betydelse den senaste observationen $d(t)$ har för nästa prognos, medan $(1-a)$ är vikten för den tidigare prognosen $u(t-1)$, som bygger på alla äldre månader. Om $a = 0$ tas ingen hänsyn till förra månadens utfall, och prognosen förblir startvärdet, 50. Om $a = 1$ bygger prognosen enbart på förra månadens utfall, så nästa månads prognos blir exakt samma värde som föregående månad.

Om $a = 0$ så kommer MAPE vara 14.0% men om $a = 1$ så kommer MAPE vara CA 20.0%. Genom att testa olika a mellan 0 till 1 så kommer MAPE:n variera mellan 14-20 %.

[2]:
```
manader =␣
 ↪["jan","feb","mar","apr","maj","jun","jul","aug","sep","okt","nov","dec"]
d = np.array([52, 51, 60, 46, 70, 50, 39, 48, 60, 70, 60], dtype=float)
u0 = 50.0

def utjamning(a, d=d, u0=u0):
    # Returnerar prognoserna u(t-1) för jan..dec (12 värden)
    u = [u0]
    for dt in d:
        u.append(a*dt + (1-a)*u[-1])
    return np.array(u)

def mape(a):
    u = utjamning(a)
    return 100*np.mean(np.abs(d - u[:-1]) / d)

alfa = np.round(np.arange(0, 1.01, 0.1), 1)
tabell = pd.DataFrame({
    "a": alfa,
    "Prognos dec": [utjamning(a)[-1] for a in alfa],
    "MAPE (%)": [mape(a) for a in alfa],
```

```
})
visa_tabell(tabell, {"a": "{:.1f}"})
```

a	Prognos dec	MAPE (%)
0.0	50.00	14.00
0.1	54.15	14.45
0.2	56.66	14.93
0.3	58.48	15.26
0.4	59.90	15.37
0.5	60.94	16.20
0.6	61.56	17.31
0.7	61.74	18.29
0.8	61.49	19.11
0.9	60.89	19.79
1.0	60.00	20.31

Härledning av den rekursiva formeln

Utgå från den viktade summan av hela tidsserien där vikterna avtar geometriskt:

$$u(t) = a\,d(t)+a(1-a)\,d(t-1)+a(1-a)^2 d(t-2)+\cdots+(1-a)^t u(0) = a\sum_{k=0}^{t-1}(1-a)^k d(t-k)+(1-a)^t u(0).$$

Skriv motsvarande uttryck för $u(t-1)$ och multiplicera med $(1-a)$:

$$(1-a)\,u(t-1) = a\sum_{k=0}^{t-2}(1-a)^{k+1} d(t-1-k) + (1-a)^t u(0) = a\sum_{k=1}^{t-1}(1-a)^k d(t-k) + (1-a)^t u(0).$$

Det är exakt alla termer i $u(t)$ utom den första, $a\,d(t)$. Alltså

$$u(t) = a\,d(t) + (1-a)\,u(t-1).$$

Vikterna $a(1-a)^k$ summerar (tillsammans med $(1-a)^t$) till 1, så det är ett riktigt viktat medelvärde där äldre observationer väger exponentiellt mindre.

1b) ABC-analys

Lagervärde per artikel = antal × medelkostnad. Artiklarna sorteras efter lagervärde och den kumulativa andelen beräknas för att göra analysen. Gränsen dras vid A = de första 80 % av värdet, B = 80-95 % och C de sista 5 %.

Lösning i Excel

Analysen är gjord i Excel och arbetsgången är följande:

1. Lagervärdet räknas ut för varje artikel och tabellen sorteras fallande efter lagervärde.
2. Lagervärdet summeras kumulativt nedåt i tabellen.
3. Den kumulativa summan delas med det totala lagervärdet, vilket ger kumulativ andel.

Kolumn	Innehåll	Formel (rad 3)
D	Lagervärde	`=B3*C3`
E	Kumulativt lagervärde	`=E2+D3`
F	Kumulativ andel	`=E3/E26`
G	ABC-klass	`=OM(F3<=0,8;"A";OM(F3<=0,95;"B";"C"))`

Lagervärdet är antal × medelkostnad. I E2 står `=D2`, och E26 är det totala lagervärdet.

En vara klassas som A om kumulativa andelen är mindre än 80 %, B om den är mindre än 95% och annars som C.

Nedan läses Excels resultat in och formlerna hämtas direkt ur filen.

[3]:
```
import openpyxl

# Värdena som Excel har räknat fram
abc = pd.read_excel("abc.xlsx", sheet_name="Blad1", header=0, usecols="A:G",␣
 ↪nrows=25)
abc.columns = ["vara", "antal", "medelkostnad", "lagervärde", "kumulativt",␣
 ↪"kumulativ", "klass"]
abc.index = range(1, len(abc) + 1)

# Formlerna som står i Excel-arket (rad 3 som exempel)
ws = openpyxl.load_workbook("abc.xlsx")["Blad1"]
for kol in "DEFG":
    print(f"{kol}3: {ws[f'{kol}3'].value}")

visa_tabell(abc, {"lagervärde": "{:,.0f}", "kumulativt": "{:,.0f}", "kumulativ":␣
 ↪"{:.2%}"})
```

```
D3: =B3*C3
E3: =E2+D3
F3: =E3/$E$26
G3: =IF(F3<=0.8,"A",IF(F3<=0.95,"B","C"))
```

vara	antal	medelkostnad	lagervärde	kumulativt	kumulativ	klass
15	84000	30	2,520,000	2,520,000	39.91%	A
19	21770	35	761,950	3,281,950	51.98%	A
13	1306	500	653,000	3,934,950	62.32%	A
3	70000	5	350,000	4,284,950	67.87%	A

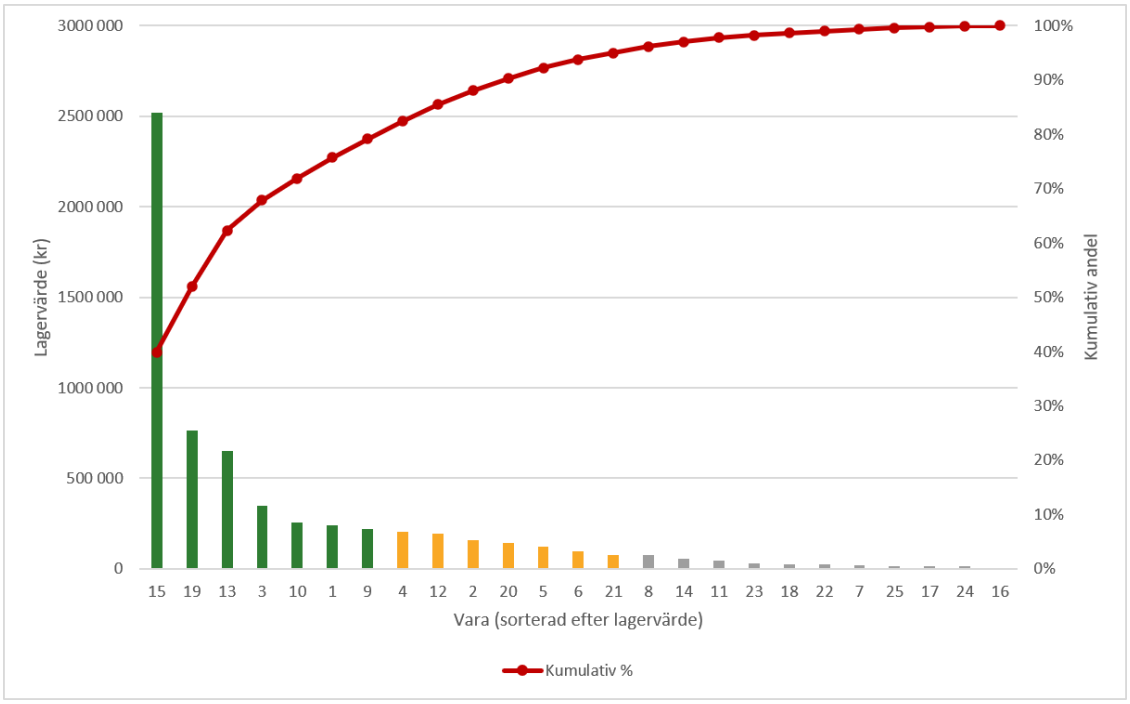
vara	antal	medelkostnad	lagervärde	kumulativt	kumulativ	klass
10	19593	13	254,709	4,539,659	71.90%	A
1	700	342	239,400	4,779,059	75.69%	A
9	10500	21	220,500	4,999,559	79.18%	A
4	68000	3	204,000	5,203,559	82.42%	B
12	19593	10	195,930	5,399,489	85.52%	B
2	800	200	160,000	5,559,489	88.05%	B
20	4200	34	142,800	5,702,289	90.31%	B
5	8638	14	120,932	5,823,221	92.23%	B
6	5355	18	96,390	5,919,611	93.76%	B
21	3200	24	76,800	5,996,411	94.97%	B
8	1306	58	75,748	6,072,159	96.17%	C
14	2800	19	53,200	6,125,359	97.01%	C
11	1400	34	47,600	6,172,959	97.77%	C
23	310	98	30,380	6,203,339	98.25%	C
18	861	27	23,247	6,226,586	98.62%	C
22	230	100	23,000	6,249,586	98.98%	C
7	10500	2	21,000	6,270,586	99.32%	C
25	1200	13	15,600	6,286,186	99.56%	C
17	840	16	13,440	6,299,626	99.78%	C
24	1000	12	12,000	6,311,626	99.97%	C
16	315	7	2,205	6,313,831	100.00%	C

[4]:
```
sammanfattning = abc.groupby("klass").agg(
    antal_artiklar=("vara", "count"),
    lagervärde=("lagervärde", "sum"),
)
sammanfattning["andel artiklar (%)"] = 100 * sammanfattning["antal_artiklar"] /␣
↪len(abc)
sammanfattning["andel lagervärde (%)"] = 100 * sammanfattning["lagervärde"] /␣
↪abc["lagervärde"].sum()
visa_tabell(sammanfattning.reset_index(), {"lagervärde": "{:,.0f}"})
```

klass	antal_artiklar	lagervärde	andel artiklar (%)	andel lagervärde (%)
A	7	4,999,559	28.00	79.18
B	7	996,852	28.00	15.79
C	11	317,420	44.00	5.03

Det blev 7 A-artiklar som står för ca 79% av hela lagervärdet. Vilket kommer att innebära tät uppföljning, noggranna prognoser och beställningar för just dessa varor. B och speciellt C-artiklar klarar sig med enkla regler, större partier och mindre kontroll.

Här i figur 1 ser vi det illustrerat hur bara 7 varor står för nästan 80 % av lagervärdet.



Figur 1. Paretodiagram över ABC-analysen, gjort i Excel.

---

2. Heltalsprogrammering

Fyra personer kan utföra sex arbetsuppgifter. Person i som gör uppgift j ger vinsten V_{ij}. Varje uppgift får göras av högst en person. Varje person har ett begränsat antal dagar, och alla uppgifter tar lika lång tid för en och samma person.

2a) Generell matematisk modell

Mängder

- $I = \{1, 2, 3, 4\}$: personer
- $J = \{1, 2, 3, 4, 5, 6\}$: arbetsuppgifter

Parametrar

- V_{ij}: vinst om person i utför uppgift j (kr)
- t_i: antal dagar det tar för person i att utföra en uppgift, $t = (2, 3, 3, 4)$
- T_i: antal tillgängliga dagar för person i, $T = (4, 4, 8, 8)$

Beslutsvariabler

$$x_{ij} = \begin{cases} 1 & \text{om person } i \text{ utför uppgift } j \\ 0 & \text{annars} \end{cases}$$

Modell

$$\max\ Z = \sum_{i \in I} \sum_{j \in J} V_{ij}\, x_{ij}$$

då

$$\sum_{i \in I} x_{ij} \leq 1 \qquad \forall j \in J \qquad \text{(varje uppgift görs av högst en person)}$$

$$\sum_{j \in J} t_i\, x_{ij} \leq T_i \qquad \forall i \in I \qquad \text{(personens tillgängliga tid)}$$

$$x_{ij} \in \{0, 1\} \qquad \forall i \in I,\ j \in J$$

Det är ett generaliserat tilldelningsproblem, en person kan få flera uppgifter, så länge tiden räcker. Det är ett binärt heltalsproblem (BIP).

Lösning

Modellen löses här i Python med `linprog`, på samma sätt som i labb 4. Variablerna ligger i ordningen $x_{11}, \ldots, x_{16}, x_{21}, \ldots, x_{46}$ (person för person). Vinsterna läggs i `obj` med minustecken eftersom `linprog` minimerar, och varje bivillkor blir en rad i `Aub` och ett tal i `Bub`.

[5]:
```
# x11 ... x16 (person 1), x21 ... x26 (person 2), x31 ... x36 (person 3), x41 ...
 ↪ x46 (person 4)

obj = [-37000, -23000, -20000, -18000, -16000,  -6000,     # person 1
       -25000, -14000, -14000, -18000, -17000,  -8000,     # person 2
        -5000, -13000, -10000, -12000,  -5000,  -5000,     # person 3
       -10000,  -3000, -13000,  -7000,  -2000, -16000]     # person 4

Aub = [[1,0,0,0,0,0,  1,0,0,0,0,0,  1,0,0,0,0,0,  1,0,0,0,0,0],    # uppgift 1
       [0,1,0,0,0,0,  0,1,0,0,0,0,  0,1,0,0,0,0,  0,1,0,0,0,0],    # uppgift 2
       [0,0,1,0,0,0,  0,0,1,0,0,0,  0,0,1,0,0,0,  0,0,1,0,0,0],    # uppgift 3
       [0,0,0,1,0,0,  0,0,0,1,0,0,  0,0,0,1,0,0,  0,0,0,1,0,0],    # uppgift 4
       [0,0,0,0,1,0,  0,0,0,0,1,0,  0,0,0,0,1,0,  0,0,0,0,1,0],    # uppgift 5
       [0,0,0,0,0,1,  0,0,0,0,0,1,  0,0,0,0,0,1,  0,0,0,0,0,1],    # uppgift 6
       [2,2,2,2,2,2,  0,0,0,0,0,0,  0,0,0,0,0,0,  0,0,0,0,0,0],    # tid person 1
       [0,0,0,0,0,0,  3,3,3,3,3,3,  0,0,0,0,0,0,  0,0,0,0,0,0],    # tid person 2
       [0,0,0,0,0,0,  0,0,0,0,0,0,  3,3,3,3,3,3,  0,0,0,0,0,0],    # tid person 3
       [0,0,0,0,0,0,  0,0,0,0,0,0,  0,0,0,0,0,0,  4,4,4,4,4,4]]    # tid person 4

Bub = [1, 1, 1, 1, 1, 1,  4, 4, 8, 8]

bnd = [(0, 1)] * 24        # varje x mellan 0 och 1
heltal = [1] * 24          # och heltal, alltså 0 eller 1

resultat = linprog(c=obj, A_ub=Aub, b_ub=Bub, bounds=bnd, integrality=heltal)
```

```
print(resultat)
```

```
        message: Optimization terminated successfully. (HiGHS Status 7: Optimal)
        success: True
         status: 0
            fun: -118000.0
              x: [ 1.000e+00  1.000e+00 ...  0.000e+00  1.000e+00]
            nit: 13
          lower:  residual: [ 1.000e+00  1.000e+00 ...  0.000e+00
                              1.000e+00]
                 marginals: [ 0.000e+00  0.000e+00 ...  0.000e+00
                              0.000e+00]
          upper:  residual: [ 0.000e+00  0.000e+00 ...  1.000e+00
                              0.000e+00]
                 marginals: [ 0.000e+00  0.000e+00 ...  0.000e+00
                              0.000e+00]
          eqlin:  residual: []
                 marginals: []
        ineqlin:  residual: [ 0.000e+00  0.000e+00  0.000e+00  0.000e+00
                              0.000e+00  0.000e+00  0.000e+00  1.000e+00
                              5.000e+00  0.000e+00]
                 marginals: [ 0.000e+00  0.000e+00  0.000e+00  0.000e+00
                              0.000e+00  0.000e+00  0.000e+00  0.000e+00
                              0.000e+00  0.000e+00]
 mip_node_count: 1
 mip_dual_bound: -118000.0
        mip_gap: 0.0
```

[6]:
```
x_opt = resultat.x
Z_opt = -resultat.fun

print(f"vinst {Z_opt}")
print(np.round(x_opt).astype(int).reshape(4, 6))      # rad = person, kolumn =␣
↪uppgift
```

```
vinst 118000.0
[[1 1 0 0 0 0]
 [0 0 0 0 1 0]
 [0 0 0 1 0 0]
 [0 0 1 0 0 1]]
```

2b) Person 1 får 6 dagar

Enda ändringen i modellen är högerledet i tidsbivillkoret för person 1. Nu hinner person 1 göra 3 uppgifter.

[7]:
```
Bub_ny = [1, 1, 1, 1, 1, 1,  6, 4, 8, 8]
```

```
resultat_ny = linprog(c=obj, A_ub=Aub, b_ub=Bub_ny, bounds=bnd,␣
 ↪integrality=heltal)

x_opt_ny = resultat_ny.x
Z_opt_ny = -resultat_ny.fun

print(f"vinst {Z_opt_ny}")
print(np.round(x_opt_ny).astype(int).reshape(4, 6))
```

```
vinst 125000.0
[[1 1 1 0 0 0]
 [0 0 0 0 1 0]
 [0 0 0 1 0 0]
 [0 0 0 0 0 1]]
```

---

3. Omlastningsmodell: probabilistiska bivillkor och heltal

Fyra bruk (A, B, C, D) skickar varor via två omlastningsnoder (E, F) till tre kunder (G, H, I). Bruken kan leverera högst 400, 350, 300 och 450 enheter. Kundernas efterfrågan är osäker och normalfördelad:

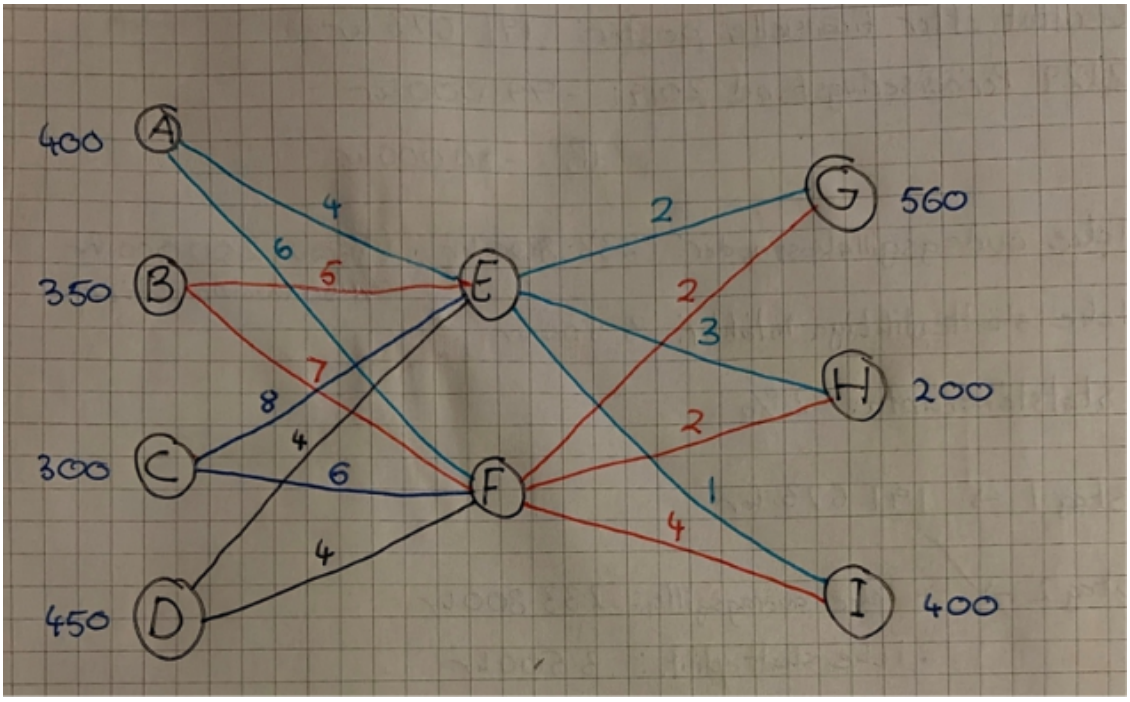
Kund	Medelvärde	Standardavvikelse
G	560	40
H	200	20
I	400	30

Transportkostnad (kr/enhet):

Från/till	E	F
A	4	6
B	5	7
C	8	6
D	4	4

Från/till	G	H	I
E	2	3	1
F	2	2	4

3a) Nätverk och definitioner



Mängder:

- Bruk (supplynoder): $S = \{A, B, C, D\}$
- Omlastningsnoder: $T = \{E, F\}$
- Kunder (demandnoder): $K = \{G, H, I\}$

Parametrar:

- s_i = maximalt utbud i bruk i: $s = (400, 350, 300, 450)$
- μ, σ = medelvärde och standardavvikelse för efterfrågan hos kund
- c_{ij} = transportkostnad per enhet från i till j

Beslutsvariabler:

- x_{ij} = antal enheter som skickas från nod i till j, till exempel x_{AE} från A till E

Det finns 8 länkar från bruken till omlastningen och 6 från omlastningen till kunderna, alltså 14 variabler: $x_{AE}, x_{AF}, x_{BE}, x_{BF}, x_{CE}, x_{CF}, x_{DE}, x_{DF}, x_{EG}, x_{EH}, x_{EI}, x_{FG}, x_{FH}, x_{FI}$.

3b) Generell modell

Målfunktion: minimera den totala transportkostnaden

$$\begin{aligned}\text{Min } Z = {} & 4x_{AE} + 6x_{AF} + 5x_{BE} + 7x_{BF} + 8x_{CE} + 6x_{CF} + 4x_{DE} + 4x_{DF} \\ & + 2x_{EG} + 3x_{EH} + x_{EI} + 2x_{FG} + 2x_{FH} + 4x_{FI}\end{aligned}$$

Bivillkor:

Bruken kan inte skicka mer än de har:

$$x_{AE} + x_{AF} \leq 400 \qquad x_{BE} + x_{BF} \leq 350 \qquad x_{CE} + x_{CF} \leq 300 \qquad x_{DE} + x_{DF} \leq 450$$

Allt som kommer in till en omlastningsnod ska skickas vidare, vilket vi kallar flödesbalans:

$$x_{AE} + x_{BE} + x_{CE} + x_{DE} = x_{EG} + x_{EH} + x_{EI}$$

$$x_{AF} + x_{BF} + x_{CF} + x_{DF} = x_{FG} + x_{FH} + x_{FI}$$

Kunderna ska få minst sin efterfrågan d_k:

$$x_{EG} + x_{FG} \geq d_G \qquad x_{EH} + x_{FH} \geq d_H \qquad x_{EI} + x_{FI} \geq d_I$$

$$x_{ij} \geq 0$$

Generellt:

$$\text{Min } Z = \sum_{(i,j)} c_{ij}\, x_{ij}, \qquad \sum_{j \in T} x_{ij} \leq s_i \ \ (i \in S), \qquad \sum_{i \in S} x_{it} = \sum_{k \in K} x_{tk} \ \ (t \in T), \qquad \sum_{t \in T} x_{tk} \geq d_k \ \ (k \in K)$$

3c) Lösning med medelvärdet som efterfrågan

Här är $d = (560, 200, 400)$.

```
MODEL:

!våra parametrar och variabler;
MIN = 4*xAE + 6*xAF + 5*xBE + 7*xBF + 8*xCE + 6*xCF + 4*xDE + 4*xDF
+ 2*xEG + 3*xEH + 1*xEI + 2*xFG + 2*xFH + 4*xFI;

! våra constrains;
xAE + xAf <= 400;
xBE + xBF <= 350;
xCE + xCF <= 300;
xDE + xDF <= 450;

!varje mellanhand delar ut lika mycket de får in;
xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

!efterfrågan är mindre än tillgången, därför måste efterfrågan uppfyllas exakt;
xEG + xFG = 560;
xEH + xFH = 200;
xEI + xFI = 400;

END
```

Variable	Value	Reduced Cost
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	310.0000	0.000000
XBF	0.000000	2.000000
XCE	0.000000	3.000000
XCF	0.000000	1.000000
XDE	250.0000	0.000000
XDF	200.0000	0.000000
XEG	560.0000	0.000000
XEH	0.000000	1.000000
XEI	400.0000	0.000000
XFG	0.000000	0.000000
XFH	200.0000	0.000000
XFI	0.000000	3.000000

Den minsta kostnaden blir 6870 kr. Bruk C används inte alls $x_{CE} = x_{CF} = 0$, eftersom det har de högsta transportkostnaderna och de andra bruken räcker för att täcka efterfrågan, 1160 enheter mot 1200 i utbud utan C. C behövs alltså inte i den optimala lösningen.

3d) Probabilistiska bivillkor

Efterfrågan är osäker, så vi kräver att varje kund ska få sin efterfrågan uppfylld med minst 90 % sannolikhet:

$$P(x_{EG} + x_{FG} \geq D_G) \geq 0{,}90$$

Med standardiserad normalfördelning blir motsvarigheten

$$x_{EG} + x_{FG} \geq \mu_G + z_{0{,}90}\,\sigma_G, \qquad z_{0{,}90} = 1{,}2816,$$

alltså leveransen ska vara medelvärdet plus 1,2816 standardavvikelser. Det enda som ändras i modellen är högerleden för efterfrågan.

```
MODEL:

!våra parametrar och variabler;
MIN = 4*xAE + 6*xAF + 5*xBE + 7*xBF + 8*xCE + 6*xCF + 4*xDE + 4*xDF
+ 2*xEG + 3*xEH + 1*xEI + 2*xFG + 2*xFH + 4*xFI;

! våra constrains;
xAE + xAf <= 400;
xBE + xBF <= 350;
xCE + xCF <= 300;
xDE + xDF <= 450;

!varje mellanhand delar ut lika mycket de får in;
xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

!efterfrågan är mindre än tillgången, därför måste efterfrågan uppfyllas exakt;
xEG + xFG >= 560 + 1.2816*40;
xEH + xFH >= 200 + 1.2816*20;
xEI + xFI >= 400 + 1.2816*30;

END
```

Och LINGO visar då detta:

Variable	Value	Reduced Cost
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	350.0000	0.000000
XBF	0.000000	2.000000
XCE	0.000000	2.000000
XCF	75.34400	0.000000
XDE	299.7120	0.000000
XDF	150.2880	0.000000
XEG	611.2640	0.000000
XEH	0.000000	1.000000
XEI	438.4480	0.000000
XFG	0.000000	0.000000
XFH	225.6320	0.000000
XFI	0.000000	3.000000

Högre signifikansnivå. Vi testar också 95 % och 99 %. Det enda som ändras är z-värdet i de tre efterfrågeraderna:

Signifikansnivå	z
90 %	1,2816
95 %	1,6449
99 %	2,3263

I LINGO byts de tre sista raderna ut. För 95 %:

```
xEG + xFG >= 560 + 1.6449*40;
xEH + xFH >= 200 + 1.6449*20;
xEI + xFI >= 400 + 1.6449*30;
```

Vilket resulterar till:

Variable	Value	Reduced Cost
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	350.0000	0.000000
XBF	0.000000	2.000000
XCE	0.000000	2.000000
XCF	108.0410	0.000000
XDE	325.1430	0.000000
XDF	124.8570	0.000000
XEG	625.7960	0.000000
XEH	0.000000	1.000000
XEI	449.3470	0.000000
XFG	0.000000	0.000000
XFH	232.8980	0.000000
XFI	0.000000	3.000000

Med kostnaden 7965kr.

För 99 %:

```
xEG + xFG >= 560 + 2.3263*40;
xEH + xFH >= 200 + 2.3263*20;
xEI + xFI >= 400 + 2.3263*30;
```

Vilket resulterar till:

Variable	Value	Reduced Cost
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	350.0000	0.000000
XBF	0.000000	2.000000
XCE	0.000000	2.000000
XCF	169.3670	0.000000
XDE	372.8410	0.000000
XDF	77.15900	0.000000
XEG	653.0520	0.000000
XEH	0.000000	1.000000
XEI	469.7890	0.000000
XFG	0.000000	0.000000
XFH	246.5260	0.000000
XFI	0.000000	3.000000

Med kostnaden 8435kr.

Kostnaden ökar alltså, enligt tabell X, med ökad signifikansnivå när vi närmar oss 100%.

Tabell X. Kostnader efter höjd signifikansnivå. Samt hur efterfrågan och användandet av C ändras.

Signifikansnivå	Total efterfrågan	Från bruk C	Kostnad
Medelvärde (c)	1 160	0	6 870 kr
90 %	1 275	75	7 714 kr
95 %	1 308	108	7 965 kr
99 %	1 369	169	8 435 kr

Med 90 % säkerhet ökar kostnaden från 6870 till 7714. Den totala efterfrågan som ska täckas blir 1275 enheter istället för 1160 och då räcker inte bruken A, B och D (400 + 350 + 450 = 1 200 enheter). Bruk C, som inte behövdes i uppgift c) måste nu användas och skickar ca 75 enheter via F.

3e) Fasta kostnader för bruken

Om vi utökar modellen i uppgift c):

```
MODEL:

@BIN(zA);
@BIN(zB);
@BIN(zC);
@BIN(zD);

!våra parametrar och variabler;
MIN = 4*xAE + 6*xAF + 5*xBE + 7*xBF + 8*xCE + 6*xCF + 4*xDE + 4*xDF
+ 2*xEG + 3*xEH + 1*xEI + 2*xFG + 2*xFH + 4*xFI
+ 300*zA + 200*zB + 250*zC + 225*zD;

! våra constrains;
xAE + xAf <= 400*zA;
xBE + xBF <= 350*zB;
xCE + xCF <= 300*zC;
xDE + xDF <= 450*zD;

!varje mellanhand delar ut lika mycket de får in;
xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

!efterfrågan är mindre än tillgången, därför måste efterfrågan uppfyllas exakt;
xEG + xFG = 560;
xEH + xFH = 200;
xEI + xFI = 400;

END
```

Vi får ut den minsta möjliga kostnaden 7595. Resultatet i LINGO visar:

Variable	Value	Reduced Cost
ZA	1.000000	-100.0000
ZB	1.000000	200.0000
ZC	0.000000	250.0000
ZD	1.000000	-225.0000
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	310.0000	0.000000
XBF	0.000000	2.000000
XCE	0.000000	3.000000
XCF	0.000000	1.000000
XDE	250.0000	0.000000
XDF	200.0000	0.000000
XEG	560.0000	0.000000
XEH	0.000000	1.000000
XEI	400.0000	0.000000
XFG	0.000000	0.000000
XFH	200.0000	0.000000
XFI	0.000000	3.000000

Fabrik C används alltså fortfarande inte. Alltså, om efterfrågan är medelvärdet, påverkar fasta kostnader inte strukturen i lösningen.

Om vi istället utökar modellen i uppgift d):

```
MODEL:

@BIN(zA);
@BIN(zB);
@BIN(zC);
@BIN(zD);


!våra parametrar och variabler;
MIN = 4*xAE + 6*xAF + 5*xBE + 7*xBF + 8*xCE + 6*xCF + 4*xDE + 4*xDF
+ 2*xEG + 3*xEH + 1*xEI + 2*xFG + 2*xFH + 4*xFI
+ 300*zA + 200*zB + 250*zC + 225*zD;

! våra constrains;
xAE + xAf <= 400*zA;
xBE + xBF <= 350*zB;
xCE + xCF <= 300*zC;
xDE + xDF <= 450*zD;

!varje mellanhand delar ut lika mycket de får in;
xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

!efterfrågan är mindre än tillgången, därför måste efterfrågan uppfyllas exakt;
xEG + xFG >= 560 + 1.2816*40;
xEH + xFH >= 200 + 1.2816*20;
xEI + xFI >= 400 + 1.2816*30;

END
```

Minsta möjliga kostnaden blir 8689. I LINGO visas:

Variable	Value	Reduced Cost
ZA	1.000000	-500.0000
ZB	1.000000	-150.0000
ZC	1.000000	250.0000
ZD	1.000000	-675.0000
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	350.0000	0.000000
XBF	0.000000	2.000000
XCE	0.000000	2.000000
XCF	75.34400	0.000000
XDE	299.7120	0.000000
XDF	150.2880	0.000000
XEG	611.2640	0.000000
XEH	0.000000	1.000000
XEI	438.4480	0.000000
XFG	0.000000	0.000000
XFH	225.6320	0.000000
XFI	0.000000	3.000000

Vad händer om bruk B (mill 2) kan leverera 450 i stället för 350 enheter?

Variable	Value	Reduced Cost
ZA	1.000000	-100.0000
ZB	1.000000	200.0000
ZC	0.000000	250.0000
ZD	1.000000	-225.0000
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	310.0000	0.000000
XBF	0.000000	2.000000
XCE	0.000000	3.000000
XCF	0.000000	1.000000
XDE	250.0000	0.000000
XDF	200.0000	0.000000
XEG	560.0000	0.000000
XEH	0.000000	1.000000
XEI	400.0000	0.000000
XFG	0.000000	0.000000
XFH	200.0000	0.000000
XFI	0.000000	3.000000

Vid användning av medelvärde påverkas inte kostnaden utan förblir 7595. Dessutom påverkas inte några variabler, alltså är lösningen exakt den samma.

Variable	Value	Reduced Cost
ZA	1.000000	-100.0000
ZB	1.000000	200.0000
ZC	0.000000	250.0000
ZD	1.000000	-225.0000
XAE	400.0000	0.000000
XAF	0.000000	2.000000
XBE	425.3440	0.000000
XBF	0.000000	2.000000
XCE	0.000000	3.000000
XCF	0.000000	1.000000
XDE	224.3680	0.000000
XDF	225.6320	0.000000
XEG	611.2640	0.000000
XEH	0.000000	1.000000
XEI	438.4480	0.000000
XFG	0.000000	0.000000
XFH	225.6320	0.000000
XFI	0.000000	3.000000

Vid användning av probalistisk begränsning och standardavvikelse ändras kostnaden till 8364. Dessutom ändras värden på några variabler. Vi kan nu se att fabrik C inte används i den optimala lösningen längre. Den ökade kapaciteten i fabrik B minskade alltså behovet av fler fabriker och kan därmed sänka totalkostnaden trots höga servicekrav. Denna modell är en Mixed Integer

Linear Programming, eller MILP, mer specifikt ett fixed-charge transshipment problem.

3f) Kapacitet på länkarna

Vi lägger till kapacitet på 350 på länkarna:

```
MODEL:

@BIN(zA); @BIN(zB); @BIN(zC); @BIN(zD);

MIN = 4*xAE + 6*xAF + 5*xBE + 7*xBF + 8*xCE + 6*xCF + 4*xDE + 4*xDF
    + 2*xEG + 3*xEH + 1*xEI + 2*xFG + 2*xFH + 4*xFI
    + 300*zA + 200*zB + 250*zC + 225*zD;

! utbud, bara om bruket är öppet;
xAE + xAF <= 400*zA;
xBE + xBF <= 350*zB;
xCE + xCF <= 300*zC;
xDE + xDF <= 450*zD;

! transportkapacitet mellan bruk och omlastning;
xAE <= 350; xAF <= 350;
xBE <= 350; xBF <= 350;
xCE <= 350; xCF <= 350;
xDE <= 350; xDF <= 350;

xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

! efterfrågan med 90 % sannolikhet;
xEG + xFG >= 560 + 1.2816*40;
xEH + xFH >= 200 + 1.2816*20;
xEI + xFI >= 400 + 1.2816*30;

END
```

där LINGO matar ut kostnaden till ca 8789. Länkarna A-E och B-E slår i taket på 350 och flödet måste därför ta andra dyrare länkar, vilket resulterar i den högre kostnaden.

Om kapaciteten istället ska vara samma på alla länkar och vi vill hitta den minsta kapaciteten K, matar vi detta till LINGO:

```
MODEL:

@BIN(zA); @BIN(zB); @BIN(zC); @BIN(zD);

MIN = K;

! utbud, bara om bruket är öppet;
xAE + xAF <= 400*zA;
xBE + xBF <= 350*zB;
xCE + xCF <= 300*zC;
xDE + xDF <= 450*zD;

! transportkapacitet mellan bruk och omlastning;
xAE <= K; xAF <= K;
xBE <= K; xBF <= K;
xCE <= K; xCF <= K;
xDE <= K; xDF <= K;

xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

! efterfrågan med 90 % sannolikhet;
xEG + xFG >= 560 + 1.2816*40;
xEH + xFH >= 200 + 1.2816*20;
xEI + xFI >= 400 + 1.2816*30;

END
```

Vi får då ut att K kommer att vara 162,56

Svaren är inte heltal, eftersom 90%-efterfrågan inte är heltal. Om vi istället använder denna LINGO kod för alla 14 flöden:

```
MODEL:

@BIN(zA); @BIN(zB); @BIN(zC); @BIN(zD);

MIN = 4*xAE + 6*xAF + 5*xBE + 7*xBF + 8*xCE + 6*xCF + 4*xDE + 4*xDF
    + 2*xEG + 3*xEH + 1*xEI + 2*xFG + 2*xFH + 4*xFI
    + 300*zA + 200*zB + 250*zC + 225*zD;

! utbud, bara om bruket är öppet;
xAE + xAF <= 400*zA;
xBE + xBF <= 350*zB;
xCE + xCF <= 300*zC;
xDE + xDF <= 450*zD;

! transportkapacitet mellan bruk och omlastning;
xAE <= 350; xAF <= 350;
xBE <= 350; xBF <= 350;
xCE <= 350; xCF <= 350;
xDE <= 350; xDF <= 350;

xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

! efterfrågan med 90 % sannolikhet;
xEG + xFG >= 560 + 1.2816*40;
xEH + xFH >= 200 + 1.2816*20;
xEI + xFI >= 400 + 1.2816*30;


@GIN(xAE); @GIN(xAF); @GIN(xBE); @GIN(xBF);
@GIN(xCE); @GIN(xCF); @GIN(xDE); @GIN(xDF);
@GIN(xEG); @GIN(xEH); @GIN(xEI);
@GIN(xFG); @GIN(xFH); @GIN(xFI);
END
```

Vi får då en något högre kostnad på 8802 istället.

Använder vi också heltal för att hitta våran gemensamma minsta kapaciteten K, gör vi:

```
MODEL:

@BIN(zA); @BIN(zB); @BIN(zC); @BIN(zD);

MIN = K;

! utbud, bara om bruket är öppet;
xAE + xAF <= 400*zA;
xBE + xBF <= 350*zB;
xCE + xCF <= 300*zC;
xDE + xDF <= 450*zD;

! transportkapacitet mellan bruk och omlastning;
xAE <= K; xAF <= K;
xBE <= K; xBF <= K;
xCE <= K; xCF <= K;
xDE <= K; xDF <= K;

xAE + xBE + xCE + xDE = xEG + xEH + xEI;
xAF + xBF + xCF + xDF = xFG + xFH + xFI;

! efterfrågan med 90 % sannolikhet;
xEG + xFG >= 560 + 1.2816*40;
xEH + xFH >= 200 + 1.2816*20;
xEI + xFI >= 400 + 1.2816*30;


@GIN(xAE); @GIN(xAF); @GIN(xBE); @GIN(xBF);
@GIN(xCE); @GIN(xCF); @GIN(xDE); @GIN(xDF);
@GIN(xEG); @GIN(xEH); @GIN(xEI);
@GIN(xFG); @GIN(xFH); @GIN(xFI);
END
```

och får ut att K blir 163.

Avrundning är överlag inte okej. Det kan bryta flödesbalansen, få ett flöde som ligger på 350 att gå över sin gräns eller göra att efterfrågan inte täcks om avrundningen görs nedåt.

---

4. Intermodal transportplanering

4a) Generell modell

4b) En färja (2000 lastbilar/år)

4c) Fler färjor

4d) Minskad kapacitet på vägen 2–3

---

5. Tomma godsvagnar

5a) Nätverk och modelltyp

5b) Generell modell

5c) Lösning

5d) Med och utan trängselvillkor

5e) Känslighet för P